i# LiteTest Runner Reference

This document provides a reference to the LiteTest Runner API. It includes types, structs, unions, enums, macros, and functions. Additionally, it provides a reference to LiteTest framework concepts .

Copyright (c) 2026 Paul Sinclair

License SPDX-License-Identifier: MIT

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Preface

This document is intended for LiteTest user and contributors who need a reference for the LiteTest Runner framework and API.

For a list of other LiteTest documents and the repository layout, see the LiteTest Documentation Guide (`LiteTest_Documentation_Guide.md`).

For a glossary of terms, see the LiteTest Glossary Reference (`LiteTest_Glossary_Reference.md`).

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version with the format `M.u` (Major, update).
- The **Runner** column records the LiteTest Runner API version current at the time this document version was published and is defined by its `LT_RUNNER_VERSION` macro.
- The **Test** column records the LiteTest Test API version current at the time this document version was published and is defined by its `LT_TEST_VERSION` macro.
- Both the Runner API and Test API use the version format `"M.m.p"` (Major, minor, patch).
- `M` is the same for the Document, Runner, and Test versions.

The document's update version tracks updates to this document and does not correspond to a minor or patch version. `u` increments whenever this document is updated without a change to `M`, and it resets to `0` when `M` is incremented.

Table of Contents

1. Introduction

TODO: add introduction, common rules.

2. Types

2.1. `lt_result_t`

Type for result counters.

Note: A test group function or test with a fault sets its result with the fault count set to SIZE_MAX to indicate a function-level fault.

See also: - `lt_currentresult(void)` in Customization Helper Functions. - `LT_GROUP(...)` notes on fault capture semantics and how function-level faults are represented.

```
typedef struct
{ size_t pass;
  size_t fail;
  size_t fault;
  size_t injected_fail;
  size_t injected_fault;
} lt_result_t;
```

2.2 `lt_state_t`

Type for maintaining the state of the orchestrator (main) or a test function.

```
typedef struct
{ size_t id;
  char *funcname;
  size_t current_level;
  size_t groupid;
  char *groupname;
  int isolation; // 0 none, 1 thread, 2 process.
  size_t category_id;
  size_t num_results_merged;
  size_t pass;
  size_t fail;
  size_t fault;
  size_t injected_fail;
  size_t injected_fault;
  size_t total_pass;
  size_t total_fail;
  size_t total_fault;
  size_t total_injected_fail;
  size_t total_injected_fault;
  lt_state_t parent;
  lt_state_t prev;
  lt_state_t next;
} lt_state_t;
```

3. Enums

3.1. lt_exit_code_t

Exit codes are grouped into classes: - `LT_EXIT[_*]` normal exit codes. - `LT_TEST[_]*` test outcome codes. - `LT_FATAL_USAGE[_*]` fatal usage codes (invalid arguments, invalid caller state, API misuse). - `LT_FATAL_INTERNAL[_*]` fatal internal codes (assert/invariant/internal failures). - `LT_FATAL_SYSTEM[_*]` fatal system codes (OS/runtime/resource failures).

Fatal codes indicate that the runner could not complete normal or test execution.

`LT_FATAL_USAGE`, `LT_FATAL_INTERNAL`, and `LT_FATAL_SYSTEM` are general codes for when a more specific code does not apply for the class.

```
typedef enum
{
  // 0-99: Exit codes.
    LT_EXIT_OK = 0,

  // 0-99: Test outcome codes.
    LT_TEST_PASS = 0,
    LT_TEST_FAIL = 1,
    LT_TEST_FAULT = 2,
    LT_TEST_FAIL_FAULT = 3,

  // 100-199: Fatal usage codes.
  LT_FATAL_USAGE = 100,
    LT_FATAL_USAGE_INVALID_ARG = 101,
    LT_FATAL_USAGE_INVALID_STATE = 102,

  // 200-299: Fatal internal codes.
  LT_FATAL_INTERNAL = 200,
    LT_FATAL_INTERNAL_ASSERT = 201,
    LT_FATAL_INTERNAL_INVARIANT = 202,

  // 300-399: Fatal system codes.
  LT_FATAL_SYSTEM = 300,
    LT_FATAL_SYSTEM_OPEN = 301,
    LT_FATAL_SYSTEM_READ = 302,
    LT_FATAL_SYSTEM_WRITE = 303,
    LT_FATAL_SYSTEM_FORK = 304,
    LT_FATAL_SYSTEM_THREAD = 305
} lt_exit_code_t;
```

3.1. lt_return_code_t

Return codes are for functions that return an integer (e.g., `int`) and are grouped into classes: - Success - Compare - Boolean - `LT_INVALID[_*]` invalid usage codes (invalid arguments). - `LT_SYSTEM[_*]` failed system codes (OS/runtime/resource failures).

LT_INVALID and LT_SYSTEM) are general codes for when a more specific code does not apply for the class.

```
typedef enum
{
  // Success
    LT_OK = 0,

  // Compare
    LT_LESS = -1,
    LT_EQUAL = 0,
    LT_GREATER = 1,

  // Boolean
    LT_FALSE = 0,
    LT_TRUE = 1,

  // -100 to -199: Invalid Usage.
    LT_INVALID = -100,
    LT_INVALID_ARG = -101,
    LT_INVALID_ARG_VERSION = -102,
    LT_INVALID_ARG_TOO_LONG = -103,

  // -300 to -399: Failed system call.
    LT_SYSTEM = -300,
    LT_SYSTEM_OPEN = -301,
    LT_SYSTEM_READ = -302,
    LT_SYSTEM_WRITE = -303,
    LT_SYSTEM_FORK = -304,
    LT_SYSTEM_THREAD = -305
} lt_return_code_t;
```

4. Macros for Limits

Maximum values for path length, filename length, and guard levels.

#define LT_MAX_PATH_LEN ((size_t)4096) #define LT_MAX_FILENAME_LEN ((size_t)255) #define LT_MAX_LEVEL ((size_t)32)

Note: The limit of 32 levels in unlikely to be exceeded if there are 2 or more `LT_GROUP` or `LT_TEST` macros at each level. It is expected that a level will generally have 2 or more per level.

5. Macros for the Orchestrator Function

Macros used to define or declare the Orchestrator (`main`) function

5.1. `LT_DECLARE_ORCHESTRATOR(funcname)[;]`

A semicolon is not allowed if the macro is followed by its definition `{...}` of the orchestrator function; otherwise, it is required and indicates this is a declaration and a forward-reference to the orchestrator function.

funcname: - Token specifying `main`.

Termination Exit Codes: - `LT_DECLARE_ORCHESTRATOR_ERROR`

5.2. `LT_INIT_ORCHESTRATOR(funcname, id, project, size_t maxparallel);`

funcname: - Token specifying `main`.

id: - A pointer to a string specifying only alphanumeric characters which is used to identify the orchestrator function. - Default is `"o"` *if* `id` is NULL pointer or its string is empty. - A short id (not exceeding 3 characters) is recommended. - The id must be unique for the orchestrator and test group functions; a violation causes the test runner to terminate.

project: - A token identifying the project. - The token is used to generate a default report title if one is not set by the `LT_PARSE_ARGS(...)` macro.

maxparallel: Maximum number of `LT_GROUP` and `LT_TEST` macros that are allowed to execute in parallel.

Termination Exit Codes: - `LT_INIT_ORCHESTRATOR_ERROR`

5.3. `LT_PARSE_ARGS(size_t maxargs, char *customflags, char *defaultreportfilename);`

maxargs: - The maximum number arguments in the command line. - The value must 2 or greater. - The first 2 arguments and LiteTest flags are parsed by the macro. - Additional arguments must be parsed by customization code.

customflags: - A string of custom flags (e.g., "-l–file-Q"). - The macro causes the test runner to terminate if a non-LiteTest flag is not in the string. - The actual parsing of these must be customization code.

defaultreportfilename: - A string defining a default report file name without an extension. The extension is `.md` since the report uses Markdown Document formatting. - The maximum length including a null terminator character is `MAX_FILENAME_LEN` (129),

Termination Exit Codes: - `LT_ARG_ERROR` - `LT_FLAG_ERROR`

See "Test Command Line" for details on command-line argument and flags.

5.4. `LT_OPEN_REPORT(char *title);`

title: - A string defining the report title. - If `title` is NULL or empty, a generated default report title " Test Report" is used. - The maximum length including a null terminator character is `MAX_TITLE_LEN` (129),

Termination Exit Codes: - `LT_OPEN_ERROR`

5.5. `LT_WRITE_RESULT(gtm, char *category);`

gtm: - A `LT_GROUP(...)` or `LT_TEST(„,)` macro.

category: - A pointer to a string with the name or description of the test category. - The maximum length including a null terminator character is `MAX_CATEOGRY_LEN` (129).

Termination Exit Codes: `LT_WRITE_ERROR`.

5.6. `LT_CLOSE_REPORT(char *notes);`

notes: - A string of note lines to append to the test report. - Each note line must end with `'\n'` (newline). - If `notes` is NULL or empty, there are no notes to append. - The maximum length including a null terminator character is `MAX_NOTE_LEN` (10001).

Termination Exit Codes: - `LT_CLOSE_ERROR`

The macro writes the totals, pre-defined explanatory notes, `notes`, notes from the temporary notes file, and final summary line to the test report file, and then closes the test report.

5.7. `LT_EXIT;`

Exit codes: - `LT_PASS` (0) - `LT_FAIL` (1) - `LT_FAULT` (2) - `LT_FAIL_FAULT` (3)

Termination Exit Codes: - `LT_EXIT_ERROR`

6. Macros for a Test Group Function

Macros used to define or declare a test group function.

6.1. `LT_DECLARE_GROUP(funcname)`

funcname: - Token specifying a function name other than `main`.

A semicolon is not allowed if the macro is followed by its definition `{...}` of the test group function; otherwise, it is required and indicates this is a declaration and a forward-reference to the test group function.

Termination Exit Codes: - `LT_DECLARE_GROUP_ERROR`

6.2. `LT_INIT_GROUP(funcname, id, maxparallel)`

funcname: - The same token as the specified token for funcname in the preceding `LT_DECLARE_GROUP` macro defining the containing test group function.

id: - A pointer to a string specifying only alphanumeric characters which is used to identify the test group function. - Default is `"funcname"` *if* `id` is NULL pointer or its string is empty. - A short id (not exceeding 3 characters) is recommended. - `id` must be unique for the orchestrator and test group functions; a violation causes the test runner to terminate.

maxparallel: - Maximum number of `LT_GROUP` and `LT_TEST` macros that are allowed to execute in parallel.

Termination Exit Codes: - `LT_INIT_GROUP_ERROR`

6.3. `LT_RETURN`

Termination Exit Codes: - `LT_RETURN_ERROR`

7. Macros to Execute a Test Group or a Test

- `LT_GROUP(funcname, id, [include], [isolation])[;]`
- `LT_TEST(expression, id, [include], [isolation])[;]`

A semicolon is not allowed if the macro is used as an argument to the `LT_WRITE_RESULT` macro; otherwise, it is required.

id: - A pointer to a string specifying only alphanumeric characters for the test group or test. - The value must be unique within the containing orchestrator or test group function - A short id (not exceeding 3 characters) is recommended or use the default. - If `id` is a NULL pointer or its string is empty, default is `n`, where `n` is `1` for the first `LT_GROUP` or `LT_TEST` macro where `id` is NULL or empty, and incremented by 1 for each subsequent `LT_GROUP` and `LT_TEST` macro where `id` is a NULL pointer or its string is empty. - See the `LT_ID` macro for obtaining the `id` for the next generated id.

include: - A single character token or omit (defaults to `1`): - `0` — Never execute. This is useful to disable a test (e.g., failing or faulting test for which the fix has been deferred). - `1` — Always execute (e.g., smoke tests). - `2-9` — Execute only when the `-In` flag is specified in the test command line and n >= include. - `I` — Execute only when the `-I` flag is specified in the test command line. This is useful for injecting a fail/fault for verifying behavior and report output if fails/faults were to occur. See also `LT_FAIL` and `LT_FAULT(type)` for simulating a fail or fault, respectively.

See “Test Command Line” for details on command line flags.

isolation: - Single character token or omit (defaults to 0): - `0` — Same thread. - `1` — Separate thread. - `2` — Separate process.

See “Isolation” in the LiteTest Runner Guide.

7.1. `LT_GROUP(funcname, id, [include], [isolation])[;]`

Termination Exit Codes: - `LT_GROUP_ERROR`

7.2. `LT_TEST(expression, id, [include], [isolation])[;]`

Termination Exit Codes: - `LT_TEST_ERROR`

8. Macros for Concurrent Blocks

8.1. `LT_BEGIN_CONCURRENT(blockname)`

blockname: - Token specifying the name of the block. - Token must be the same as for the subsequent matching `LT_END_CONCURRENT(blockname)`.

Termination Exit Codes: - `LT_BEGIN_GROUP_ERROR`

8.2. `LT_END_CONCURRENT(blockname);`

blockname: - Token specifying the name of the block. - Token must be the same as for the preceding matching `LT_BEGIN_CONCURRENT(blockname)`.

Termination Exit Codes: - `LT_END_GROUP_ERROR`

9. Macros for Versioning

9.1. LT_RUNNER_VERSION

Value: - LiteTest Runner's version as a literal string of type `char[n]`, where `n` is between 5 and 9 (including the terminating null character). - Format: `M.m.p`, where `M`, `m`, and `p` are one or two digits. - `M` is the major version, `m` is the minor version, and `p` is the patch version.

The major version is incremented for major additions, removal of deprecated features, or unavoidable incompatible API changes.

The minor version is incremented for backward-compatible additions or deprecating features.

The patch version is incremented for bug fixes or internal improvements.

Examples: - `LT_RUNNER_VERSION` → `"1.0.9"` - `LT_RUNNER_VERSION` → `"1.3.11"` - `LT_RUNNER_VERSION` → `"12.05.18"`

9.2. LT_VERSION_MAJOR(v)

v: pointer to a version string.

Value: - The major version in `v` cast to `int`. - Value is `LT_INVALID_VERSION` (`(int)-2`) *if* `v` is an invalid version string.

Examples: - `LT_VERSION_MAJOR("5.0.9")` → `(int)5` - `LT_VERSION_MAJOR("5.000.9")` → `LT_INVALID_VERSION`

9.3. LT_VERSION_MINOR(v)

v: pointer to a version string.

Value: - The minor version in `v` cast to `int`. - Value is `LT_INVALID_VERSION` (`(int)-2`) *if* `v` is an invalid version string.

Examples: - `LT_VERSION_MINOR("5.00.9")` → `(int)0` - `LT_VERSION_MINOR("5.1.001")` → `LT_INVALID_VERSION`

9.4. LT_VERSION_PATCH(v)

v: pointer to a version string.

Value: - The patch version in `v` cast to `int`. - Value is `LT_INVALID_VERSION` (`(int)-2`) *if* `v` is an invalid version string.

Examples: - `LT_VERSION_PATCH("5.0.12")` → `(int)12` - `LT_VERSION_PATCH("5.000.9")` → `LT_INVALID_VERSION`

9.5. LT_VERSION_NUM(v)

v: pointer to a version string.

Value: - Representation of the version `v` cast to `int`. - Value is `LT_INVALID_VERSION` (`(int)-2`) *if* `v` is an invalid version string.

Form: MMmmpp for comparisons, e.g., 10000 for version 1.0.0, 10200 for version 1.2.0, or 11212 for version 1.12.12.

Examples: - `LT_VERSION_NUM("1.0.9")` → `(int)0x010009` - `LT_VERSION_NUM("2.5.3")` → `(int)0x020503`

9.6. `LT_VERSION_HEX(v)`

v: - Pointer to a version string.

Value: - Hex representation of version `v` cast to `int`. - Value is `LT_INVALID_VERSION` ((int)`-2`) *if* `v` is an invalid version string.

Hexadecimal form 0xMMmmpp for display/debugging, e.g., 0x010000 for version 1.0.0, 0x010200 for version 1.2.0, or 0x011212 for version 1.12.12.

Examples: - `LT_VERSION_HEX("1.0.9")` → `0x010009` - `LT_VERSION_HEX("2.5.3")` → `0x020503`

9.7. `LT_VERSION_CMP(v1, v2)`

v1: - Pointer to a version string.

v2: - Pointer to a version string.

Value: - Integer (`int`) result comparing version `v1` to `v2`: - `LT_EQUAL` (`(int)0`) — equal - `LT_LESS` (`(int)-1`) — `v1` is less than `v2` - `LT_GREATER` (`(int)1`) — `v1` is greater than `v2` - `LT_INVALID_VERSION` (`(int)-2`) — invalid `v1` or invalid `v2` - Leading zeros in `M`, `m`, and `p` are ignored during comparison.

Examples: - `LT_VERSION_CMP(LT_RUNNER_VERSION, "1.10.0")` → `LT_GREATER` (*if* `LT_RUNNER_VERSION` is `"1.10.6"`) - `LT_VERSION_CMP(LT_RUNNER_VERSION, "1.10.0")` → `LT_LESS` (*if* `LT_RUNNER_VERSION` is `"1.8.0"`) - `LT_VERSION_CMP(LT_RUNNER_VERSION, "1.10.0")` → `LT_EQUAL` (*if* `LT_RUNNER_VERSION` is `"1.10.0"`) - `LT_VERSION_CMP("1.10.0x", "1.10.0")` → `LT_INVALID_VERSION` - `LT_VERSION_CMP("1.10.0", "1.10y.0")` → `LT_INVALID_VERSION`

10. Macros for Customization

10.1. `LT_PRINT_ERR_HELP(char *err, char help)`

Print err and help text to stdout. - `err` is prefixed with `lt_err_prefix()`. - Help text is formed using `lt_usage()` and `lt_help()`.

err - Pointer to error string.

help - 0: don't print help text. - non-zero: print help text.

Note: Use `lt_set_err_prefix()` to set the prefix.

Note: Use `lt_set_usage()` and/or `lt_set_help()` to define the help text.

11. Functions for Customization

11.1. `int lt_funcname(char *funcname, size_t outlen)`

11.2. `int lt_print_note(const char *notes)`

Opens a temporary file if not already open and writes the notes to the temporary file.

notes: - A string of note lines to write to the temporary file. - Each note line must end with `'\n'` (newline). - If `notes` is NULL or empty, the temporary file is removed if it exists. - The maximum length including a null terminator character is `MAX_NOTE_LEN` (10001).

Return: - `LT_SUCCESS` - `LT_NOTES_REMOVED` - `LT_NOTES_OPEN_ERROR` - `LT_PRINT_NOTE_ERROR`

`void lt_current_time(char *current_time, size_t size)`

Get the current time as a formatted string.

current_time: - Buffer to store the formatted time string.

size Size of the buffer.

Note: On error, current_time is set to "unknown time".

Customization: Get and set.

`char *lt_executable_name(void)` `void lt_set_executable_name(char *en)` `char *lt_default_dirpath(void)` `void lt_set_default_dirpath(char *dp)` `char *lt_err_prefix(void)` `void lt_set_err_prefix(char *pe)` `char *lt_args_options(void)` `void lt_set_args_options(cha *ao)` `char *lt_usage(void)` `void lt_set_usage(char *u)` `char *lt_help(void)` `void lt_set_help(char *h)`

Customization Helper Functions

`size_t lt_currentlevel(void)` `lt_result_t lt_currentresult(void)` `lt_total_t lt_currenttotal(void)` `size_t lt_maxparallel(size@_t level)` `size_t lt_currentparallel(void` `int lt_isisolated(void)` `int lt_isthreadisolated(void)` `int lt_isprocessisolated(void)` `size_t lt_groupid(void)` `char *lt_groupname(void)`

`char *lt_project(void)` `size_t lt_maxargs(void)` `char *lt_title(void)` `size_t lt_categoryid(void)` `char *lt_category(void)` `char *lt_funcname(void)` `char *lt_notes(void)` `char *lt_assertexpression(void)` `char lt_inject(void)` `char lt_isolation(void)` `char lt_orchestrator(void)` `char lt_testfunction(void)` `char lt_assert(void)`

`char *lt_dirpath(void)` `char *lt_filepath(void)` `char` *lt_filename(void)‘

Return: 1 true, 0 false `int lt_isfilename(char *name)`

Return: 1 true, 0 false, -101 invalid directory path. `int lt_isreaddirpath(char *path)` `int lt_iswritedirpath(char *path)`

Return 1 true, 0 false, -103 invalid file path. `int lt_isreadfilepath(char *path)` `int lt_iswritefilepath(char *path)`

12. Test Command Line

TODO.

13. Output and Golden Files

Output files in `tests/output/` and golden files in `tests/golden/` subdirectories of the repository root provide a directory layout for capturing and validating test output.

13.1. Output Files

A oopy of a named output file is captured to `tests/output/` : - If the output file name has the form ., the `tests/output/` file is named -.ext. - If the output file name has the form (with no extension), the `tests/output/` file is named -.

A copy of `stdout` is captured in `tests/output/` output file named `stdout-<uid>` and a copy of `stderr` is captured as an output file named `stderr-<uid>`.

`<uid>` is a unique id string for each captured output file within a test run. It is used to avoid filename collisions when the same output file name appears multiple times (e.g., tests of a command line varying options each generationg stdout and stderrj).

Temporary files are not captured in `tests/output/`.

A metadata file (in Markdown Document format) for each output file is also written to `tests/output/`: - If the output file name has the form ., the `tests/output/` metadata file is named -.ext.md. - If the output file name has the form (with no extension), the `tests/output/` file is named -.md.

An output file and its metadata file in `tests/output/` may be copied (promoted) to the `tests/golden/` subdirectory of the repository root directory. See the following section.

13.2. Golden Files

A metadata golden file having a name with the form . applies to all golden files named -.ext and can be added manually or by customization code added to the test runner.

A metadata golden file having a name with the form .md applies to all golden files named - with no extension and can be added manually or by customization code added to the test runner.

Golden files (located in the `tests/golden/`) are handled as follows:

- If the corresponding golden file with the same name does not exist,
  - If a metadata golden file .ext or exists and contains the line `##MATCH##`, comparison is skipped and a match is forced. If it contains the line `##MISMATCH##`, comparison is skipped and a mismatch is forced.
  - Otherwise, the output file is automatically copied (promoted) into the `tests/golden` directory and a match forced.
  - The test report notes that the file was promoted.
- If the corresponding golden file exists and contains the line `##MATCH##`, comparison is skipped and a match is forced.
- If the corresponding golden file exists and contains the line `##MISMATCH##`, comparison is skipped and a mismatch is forced.
- Otherwise, the output file is compared with its existing golden file of the same name in the `tests/golden` directory to determine whether they match.

If an output file does not match its golden file but inspection determines the output is correct, update the golden file with a copy of the output. Alternatively, remove the golden file so that the next run will automatically promote the output.

13.3. Output/Golden Rationale

LiteTest uses a parallel `tests/golden` directory with golden files that keep the exact same filename as their corresponding test output. This design avoids the common problems found in extension-based naming schemes:

- **No filename collisions** Different outputs such as `foo.c` and `foo.h` remain distinct (`foo-<uid>.c` vs. `foo-<uid>.h`) instead of collapsing into a single `foo.golden`.
- **No special-case rules** The golden filename is always identical to the output filename.
- **Preserves file type information** Because the original extension is retained, it is immediately clear whether a golden file contains text, Markdown Document, JSON, HTML, binary data, or something else.
- **Simpler mental model** Developers do not need to remember naming conventions or extension-replacement rules. Golden files simply live in a parallel directory and share the same name as the output they validate.